Ford One

Documentação Técnica

Portal, Painel da Concessionária e Backend

Versão 1.0 - Setembro de 2026

Visão geral

O Ford One é uma plataforma composta por três aplicações web e um backend compartilhado:

- **Portal** (ford-one/main): landing page institucional, tela de escolha de acesso, login da concessionária e formulário público de captação de leads.
- **Dealership** (ford-one/main/dealership): painel administrativo interno da concessionária, usado por colaboradores autenticados (ADMIN, GERENTE, FUNCIONARIO).
- **Backend** (ford-scrapper-service): API REST única, em NestJS, que atende tanto o portal quanto o dealership, além do pipeline de sincronização do catálogo de veículos Ford (o scraper).

As três aplicações rodam como containers Podman/Docker independentes, orquestrados via docker-compose, com um proxy Caddy fazendo a terminação TLS na frente de todas.

Arquitetura do backend

O backend usa NestJS com Prisma como ORM sobre PostgreSQL. Cada domínio de negócio é isolado em seu próprio módulo, seguindo uma arquitetura em camadas:

- **presentation/** - controllers HTTP: rotas, validação de entrada via DTOs e guards de autorização.
- **application/** - services com a lógica de negócio, além dos DTOs de entrada e saída.
- **infrastructure/** - repositories com as consultas reais ao banco via Prisma.

Módulos de negócio existentes: agendamento, audit-log, auth, avaliação, cliente, colaborador, configurações, dashboard, estoque, financiamento, lead, meta, serviço, técnico, vehicle e veículo-cliente. Módulos de infraestrutura: common (guards, interceptors e DTOs compartilhados), health (healthcheck), scheduler (jobs periódicos de ciclo de vida, ver seção seguinte), e scrapper e search, responsáveis pelo pipeline de sincronização do catálogo Ford.

Autenticação e segurança

- Login: token JWT assinado (HS256), com expiração configurável via a variável JWT_EXPIRES_IN (padrão de 8 horas). O endpoint de login tem limite de 5 tentativas a cada 60 segundos.
- Senhas: hash com argon2id (memoryCost 19456, timeCost 2, parallelism 1) - nunca armazenadas em texto puro nem de forma reversível.
- Autorização: RolesGuard global combinado ao decorator @Roles(...) em cada rota. Existem três papéis - ADMIN, GERENTE e FUNCIONARIO - e cada rota declara explicitamente quem pode acessá-la.
- Auditoria: o AuditInterceptor, aplicado globalmente, registra toda escrita (POST, PATCH, PUT, DELETE) feita em qualquer controller, com usuário, ação, recurso, status HTTP e horário. A leitura desse histórico é exposta em GET /audit-log, restrita a administradores.
- Proteções HTTP: Helmet para cabeçalhos de segurança, CORS com lista de origens permitidas (recusa curinga em produção) e rate limiting global de 60 requisições por minuto.

- Dados sensíveis: o CPF de cada colaborador é sempre mascarado nas respostas da API, e as chaves de criptografia são configuradas por variável de ambiente.

Regras automáticas de negócio

Transições de estado que uma concessionária real espera acontecer sozinhas, sem passo manual, disparadas por um evento de negócio real:

- **Lead vira Cliente automaticamente** em dois eventos reais e independentes: a aprovação de um Financiamento vinculado ao lead, ou a conclusão de uma Ordem de Serviço paga (valor maior que zero) originada de um agendamento do lead - uma simples visita ao showroom (valor zero) não converte. Em ambos os casos o sistema resolve ou cria o Cliente por e-mail/telefone (evitando duplicata) e registra o histórico se um cadastro antigo for atualizado.
- **Prioridade de Ordem de Serviço** (No prazo / Em risco / Atrasado) nunca é escolhida manualmente - é sempre calculada a partir de um prazo real (prazoData) e reavaliada automaticamente a cada hora por um job (LifecycleJobsService.reavaliarPrioridadeDasOS), para que uma OS "No prazo" não fique com essa etiqueta congelada depois do prazo vencer.
- Outros jobs periódicos do mesmo módulo: geração de Ordem de Serviço a partir de um Agendamento vencido, escalonamento de urgência de leads sem contato recente, decaimento de status de clientes inativos e liberação automática de reservas de estoque expiradas.

Dashboard e métricas

O módulo de dashboard calcula tudo a partir de dados reais das demais tabelas - não existe nenhum valor fixo ou estimado no meio do caminho:

- Indicadores principais (leads, agendamentos, receita, conversão, SLA, nota média e estoque), cada um com a variação percentual em relação ao período anterior de mesma duração, em 5 períodos selecionáveis: hoje, semana, mês, trimestre e ano.
- Série de receita e conversão de leads com granularidade adaptada ao período selecionado - blocos de 2h (hoje), por dia (semana), por semana (mês) ou por mês (trimestre: 6 meses; ano: 12 meses). No período Ano, a linha de meta é substituída por uma comparação direta com o ano anterior (colunas pareadas, mês a mês).
- Quatro alertas operacionais calculados em tempo real: SLA em risco, estoque baixo, lead prioritário e meta atingida.
- Endpoint /dashboard/receita: receita já confirmada versus receita em pipeline, com a quebra por categoria (financiamentos e serviços) e a série trimestral.
- Endpoint /dashboard/desempenho: retenção de clientes, produtividade das ordens de serviço e o ranking de conversão por consultor, calculado a partir de Lead.responsavelId.

Painel da concessionária (dealership)

É uma SPA construída em React, TypeScript e Vite. Seus módulos são: dashboard, clientes, histórico, estoque, leads, agenda, serviços, metas, financiamentos, receita, avaliação, desempenho, relatórios e

configurações. Cada um segue o mesmo padrão interno: uma pasta services/ para as chamadas HTTP, uma pasta hooks/ para estado e lógica, e um arquivo <módulo>.screen.tsx para a apresentação. A tela de Configurações reúne, em abas: os dados cadastrais da concessionária; a gestão de acesso da equipe, com criação, edição e desativação de colaboradores (a criação é restrita a administradores); um resumo das metas; e uma aba Sistema com aparência (tema claro ou escuro), o status de segurança acompanhado da atividade recente do log de auditoria (lista com rolagem própria, limitada a 8 entradas), e esta própria documentação.

Portal público

O portal reúne a landing page institucional, a tela de escolha do tipo de acesso, o login da concessionária - que autentica contra o mesmo backend e repassa o token JWT ao dealership por um fragmento de URL - e o formulário público de captação de leads, que grava diretamente na tabela Lead e dispara uma notificação por e-mail via Resend.

Infraestrutura e deploy

- Containers: postgres (imagem 17-alpine, com volume nomeado e persistente), backend, portal, dealership e caddy.
- Build multi-stage: Node para compilar cada aplicação, e nginx alpine para servir os SPAs já compilados em produção.
- TLS via Caddy - em ambiente local usa um certificado autoassinado interno; ao apontar um domínio real, basta trocar o host no Caddyfile e remover a diretiva de certificado interno para que a emissão via Let's Encrypt aconteça automaticamente.
- Backups do banco por dump agendado, guardados na pasta backups/.
- Migrações de schema aplicadas via Prisma, com o comando não interativo prisma migrate deploy.

Convenções de projeto

- Nunca exibir dado inventado na interface: um campo sem fonte real fica oculto ou mostra um estado vazio explícito, nunca um placeholder disfarçado de dado verdadeiro.
- Paginação real em toda listagem, através dos tipos compartilhados PaginationQueryDto e PaginationMeta.
- Erros de API são traduzidos para uma mensagem amigável no frontend pela função friendlyApiMessage.
- Toda mutação (POST, PATCH ou DELETE) passa automaticamente pelo AuditInterceptor - isso não é uma opção configurável, é parte estrutural do sistema.
- Campos de FK (ex.: clienteId, tecnicoId, estoqueVeiculoId) sempre resolvem o texto de exibição a partir do cadastro real quando informados - o texto solto é só um cache de leitura, nunca a fonte da verdade.